

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

Q1
Runtime Polymorphism in
Sustainability Dashboards
10 marks**Q2**
Hybrid Design in Smart
Agriculture
10 marks**Q3**
Function Overriding in Green
Transport
10 marks**Q4**
Virtual Base Class for Battery
Data
10 marks**Q5**
Method Chaining with this
Pointer
10 marks

5/5 answered

Q1 Runtime Polymorphism in Sustainability Dashboards

10 marks

A sustainability dashboard processes SolarPanel, WindTurbine and Battery objects using a common base-class pointer. Analyze how runtime polymorphism supports different energy calculations.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class EnergySource {
5 public:
6     virtual double calculateEnergy() = 0;
7 };
8
9 class SolarPanel : public EnergySource {
10 public:
11     double calculateEnergy() override {
12         return 500; // Solar energy
13     }
14 };
15
16 class WindTurbine : public EnergySource {
17 public:
18     double calculateEnergy() override {
19         return 800; // Wind energy
20     }
21 };
22
23 class Battery : public EnergySource {
24 public:
25     double calculateEnergy() override {
26         return 300; // Stored battery energy
27     }
28 };
29
30 int main() {
31     EnergySource* source;
32
33     SolarPanel solar;
34     WindTurbine wind;
35     Battery battery;
36
37     source = &solar;
38     cout << "Solar Energy: " << source->calculateEnergy() << endl;
39
40     source = &wind;
41     cout << "Wind Energy: " << source->calculateEnergy() << endl;
42
43     source = &battery;
44     cout << "Battery Energy: " << source->calculateEnergy() << endl;
45
46     return 0;
47 }
```

Input

stdin input

Output

Visible Test Cases

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

Q1
Runtime Polymorphism in
Sustainability Dashboards
10 marks**Q2**
Hybrid Design in Smart
Agriculture
10 marks**Q3**
Function Overriding in Green
Transport
10 marks**Q4**
Virtual Base Class for Battery
Data
10 marks**Q5**
Method Chaining with this
Pointer
10 marks

5/5 answered

Q2 Hybrid Design in Smart Agriculture

10 marks

A smart agriculture system uses inheritance for different farm types and composition for sensors. Analyze both approaches and implement a hybrid sustainable-farming system.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 // Sensor classes
5 class SoilSensor {
6 public:
7     void readSoilMoisture() {
8         cout << "Soil Moisture: 65%" << endl;
9     }
10 };
11
12 class TemperatureSensor {
13 public:
14     void readTemperature() {
15         cout << "Temperature: 28 C" << endl;
16     }
17 };
18
19 // Base Farm class
20 class Farm {
21 protected:
22     SoilSensor soilSensor;
23     TemperatureSensor temperatureSensor;
24
25 public:
26     virtual void farmingMethod() {
27         cout << "General sustainable farming" << endl;
28     }
29
30     void monitorSensors() {
31         soilSensor.readSoilMoisture();
32         temperatureSensor.readTemperature();
33     }
34
35     virtual ~Farm() {}
36 };
37
38 // Derived farm classes
39 class OrganicFarm : public Farm {
40 public:
41     void farmingMethod() override {
42         cout << "Organic farming using natural fertilizers" << endl;
43     }
44 };
45
46 class SmartFarm : public Farm {
47 public:
48     void farmingMethod() override {
49         cout << "Smart farming using automated irrigation" << endl;
50     }
51 };
52
53 int main() {
54     OrganicFarm organic;
55     SmartFarm smart;
56
57     cout << "Organic Farm:" << endl;
58     organic.farmingMethod();
59     organic.monitorSensors();
60
61     cout << "\nSmart Farm:" << endl;
62     smart.farmingMethod();
63     smart.monitorSensors();
64
65     return 0;
66 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

Q1
Runtime Polymorphism in
Sustainability Dashboards
10 marks**Q2**
Hybrid Design in Smart
Agriculture
10 marks**Q3**
Function Overriding in Green
Transport
10 marks**Q4**
Virtual Base Class for Battery
Data
10 marks**Q5**
Method Chaining with this
Pointer
10 marks

5/5 answered

Q3 Function Overriding in Green Transport

10 marks

A green-transport platform calculates environmental impact differently for ElectricBus and DieselBus. Analyze how function overriding improves runtime behavior.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Bus {
5 public:
6     virtual void calculateImpact() {
7         cout << "Calculating general environmental impact" << endl;
8     }
9
10    virtual ~Bus() {}
11 };
12
13 class ElectricBus : public Bus {
14 public:
15     void calculateImpact() override {
16         cout << "Electric Bus: Very low carbon emissions" << endl;
17     }
18 };
19
20 class DieselBus : public Bus {
21 public:
22     void calculateImpact() override {
23         cout << "Diesel Bus: Higher carbon emissions" << endl;
24     }
25 };
26
27 int main() {
28     Bus* bus;
29
30     ElectricBus electricBus;
31     DieselBus dieselBus;
32
33     bus = &electricBus;
34     bus->calculateImpact();
35
36     bus = &dieselBus;
37     bus->calculateImpact();
38
39     return 0;
40 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

Q1 ✓
Runtime Polymorphism in
Sustainability Dashboards
10 marks**Q2** ✓
Hybrid Design in Smart
Agriculture
10 marks**Q3** ✓
Function Overriding in Green
Transport
10 marks**Q4** ✓
Virtual Base Class for Battery
Data
10 marks**Q5** ✓
Method Chaining with this
Pointer
10 marks

5/5 answered

Q4 Virtual Base Class for Battery Data

10 marks

An EV battery-management system inherits common battery information through multiple classes, causing duplicate data. Analyze the issue and resolve it using a virtual base class.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Battery {
5 protected:
6     int id;
7     float capacity;
8
9 public:
10     Battery(int i, float c) {
11         id = i;
12         capacity = c;
13     }
14
15     void display() {
16         cout << "Battery ID: " << id << endl;
17         cout << "Capacity: " << capacity << " kWh" << endl;
18     }
19 };
20
21 class Charging : virtual public Battery {
22 public:
23     Charging(int i, float c) : Battery(i, c) {}
24
25     void charge() {
26         cout << "Charging system active" << endl;
27     }
28 };
29
30 class Monitoring : virtual public Battery {
31 public:
32     Monitoring(int i, float c) : Battery(i, c) {}
33
34     void monitor() {
35         cout << "Monitoring system active" << endl;
36     }
37 };
38
39 class EVBattery : public Charging, public Monitoring {
40 public:
41     EVBattery(int i, float c)
42         : Battery(i, c), Charging(i, c), Monitoring(i, c) {}
43
44     void show() {
45         display();
46         charge();
47         monitor();
48     }
49 };
50
51 int main() {
52     EVBattery b(101, 75.5);
53
54     b.show();
55
56     return 0;
57 }
```

Input

stdin input

Output

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

Q1
Runtime Polymorphism in
Sustainability Dashboards
10 marks

Q2
Hybrid Design in Smart
Agriculture
10 marks

Q3
Function Overriding in Green
Transport
10 marks

Q4
Virtual Base Class for Battery
Data
10 marks

Q5
Method Chaining with this
Pointer
10 marks

5/5 answered

Q5 Method Chaining with this Pointer

10 marks

A carbon-tracking application supports method chaining for updating energy and emission values. Analyze how the this pointer enables chaining.

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class CarbonTracker {
5 private:
6     double energy;
7     double emissions;
8
9 public:
10    CarbonTracker() {
11        energy = 0;
12        emissions = 0;
13    }
14
15    CarbonTracker& setEnergy(double e) {
16        energy = e;
17        return *this;
18    }
19
20    CarbonTracker& setEmissions(double e) {
21        emissions = e;
22        return *this;
23    }
24
25    CarbonTracker& addEnergy(double e) {
26        energy += e;
27        return *this;
28    }
29
30    void display() {
31        cout << "Energy: " << energy << " kWh" << endl;
32        cout << "Emissions: " << emissions << " kg CO2" << endl;
33    }
34 };
35
36 int main() {
37    CarbonTracker tracker;
38
39    tracker.setEnergy(100)
40        .setEmissions(25)
41        .addEnergy(50)
42        .display();
43
44    return 0;
45 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1